



CAPÍTULO 8

Cadeias de caracteres

Como o C representa texto sem ter um tipo `string` dedicado — e o que isso muda na forma de ler, comparar e guardar palavras.

R i o \0

O que vamos ver

01 Caracteres

char é um número — a tabela ASCII por trás de cada letra

02 Cadeias de caracteres

o papel do '\0', como ler e como manipular uma string

03 Vetor de strings

um vetor de ponteiros, não uma grade fixa de caracteres

04 Parâmetros de main

como um programa lê o que foi digitado depois do seu nome

Um char é só um número pequeno

Escrever 'a' no código é uma forma legível de escrever 97 — por isso dá pra somar, comparar, ordenar.



A tabela ASCII

	0	1	2	3	4	5	6	7	8	9
30			sp	!	"	#	\$	%	&	'
40	(	)	*	+	,	-	.	/	0	1
50	2	3	4	5	6	7	8	9	:	;
60	<	=	>	?	@	A	B	C	D	E
70	F	G	H	I	J	K	L	M	N	O
80	P	Q	R	S	T	U	V	W	X	Y
90	Z	[	\	]	^	_	`	a	b	c
100	d	e	f	g	h	i	j	k	l	m
110	n	o	p	q	r	s	t	u	v	w
120	x	y	z	{		}	~			

Figura 8.1: Códigos ASCII de alguns caracteres que podem ser impressos (sp representa espaço em branco).

Nem todo código desenha algo

Caracteres de controle comandam — '`\0`' marca o fim de uma string, '`\n`' pula de linha.

0	<i>null</i> : nulo
7	<i>bell</i> : campainha
8	<i>backspace</i> : volta e apaga um caractere
9	<i>tab</i> : tabulação horizontal
10	<i>newline</i> ou <i>line feed</i> : muda de linha
13	<i>carriage return</i> : volta ao início da linha
127	<i>delete</i> : apaga um caractere

Figura 8.2: Códigos ASCII de alguns caracteres de controle.

Um vetor de char, mais uma convenção

R i o \0

É o `'\0'` — não o tamanho do vetor — que diz onde a string termina.

Nem todo scanf lê o que parece

Entrada digitada: Hello World

"%s"

Para no primeiro espaço em branco.

```
HelloWorld
```

"%c"

Lê um único caractere — útil só quando é isso mesmo que se quer.

```
HelloWorld
```

" %99[^\n]"

Lê a linha inteira, espaços incluídos, até o Enter.

```
Hello World
```

O buffer lembra o que sobrou

Usuário digita 20 e pressiona Enter —
`scanf("%d", ...)` lê o número, mas não o
Enter.

```
scanf("%d", &idade);  
scanf("%c", &letra);
```

SOBROU NO BUFFER

`\n`

✗ letra = '\n' — não esperou o usuário digitar nada

```
scanf("%d", &idade);  
scanf(" %c", &letra);
```

SOBROU NO BUFFER

`\n`

✓ o espaço descarta o '\n' e espera um novo caractere

O mesmo laço, quatro tarefas

Todas varrem um vetor de char até encontrar o `'\0'`.

strlen

quantos caracteres antes do `'\0'`

strcpy

copia até o `'\0'` (inclusive)

strcat

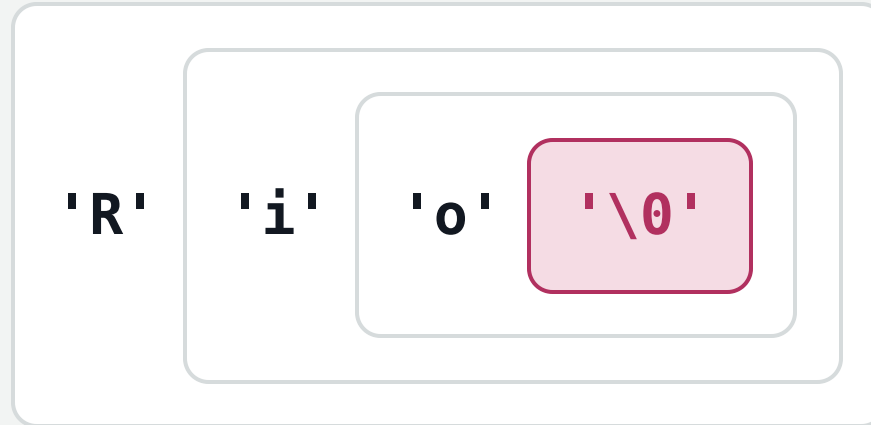
anexa uma string ao fim de outra

strcmp

compara caractere a caractere

Uma outra forma de enxergar

Uma string é a string vazia, ou um caractere seguido de uma string (o resto).



Aspas simples $\neq$ aspas duplas

R

'R' – 1 byte, tipo char

VS

R

\0

"R" – 2 bytes, tipo string

Vetor de strings = vetor de ponteiros

Cada string pode ter um tamanho diferente —
cada posição só guarda o endereço da sua.

[0] nomes[0] →

[1] nomes[1] →

[2] nomes[2] →

"Ana"

"Bruno"

"Carla"

main também recebe argumentos

./main ola mundo

./main

ola

mundo

argv[0]

argv[1]

argv[2]

```
06_COMMAND_LINE_ARGS/MAIN.C
int main(int argc, char *argv[])
{
    printf("Numero de argumentos (argc): %d\n", argc);

    for (int i = 0; i < argc; i++)
    {
        printf("argv[%d] = \"%s\"\n", i, argv[i]);
    }

    return 0;
}
```

AGORA É COM VOCÊ

Explore os exemplos, no seu ritmo

Código-fonte, explicação linha a linha e simulação da execução passo a passo — sem precisar compilar nada. Depois, teste o que aprendeu no quiz.



Notas de Aula e Exemplos thiagopx.github.io/ed/06_strings